Software Requirements Specification (SRS)

System Title: Use Case-3: Hotel Booking System

Team Name: Future Builders

Frontend: Parthiban, Yugendra

Backend: MahaLakshmi, Chaithra

📖 1. Introduction

1.1 Purpose

The purpose of this system is to provide a seamless platform for users to search, explore, and book hotel rooms online. It enables users to filter hotels based on location, price, and amenities, and manage bookings efficiently.

1.2 Scope

The system is a full-stack web application built using:

- **Frontend**: React (Vite)
- **Backend**: Spring Boot
- **Database**: Relational DB (MySQL/PostgreSQL)

It supports:

- User authentication and authorization
- Hotel and room management
- Booking system
- Amenity-based filtering
- Admin control features

🎯 2. Overall Description

2.1 Product Perspective

The system is a standalone web application with:

- REST APIs for backend communication
- Responsive frontend UI

- Secure authentication system

2.2 User Classes

Admin

- Manage hotels, rooms, amenities
- View all bookings

User (Customer)

- Register/login
- Search hotels
- Book rooms
- View bookings

2.3 Operating Environment

- **Web browser**: Chrome, Edge, Firefox, Safari
- **Backend server**: Spring Boot JVM Environment
- **Database server**: MySQL / PostgreSQL
- **Port Setup**:
    - **Frontend Port**: 5173 (Vite Default)
    - **Backend Port**: 8081 (Spring Boot Default)

⚙️ 3. Functional Requirements

🔐 3.1 Authentication

- User registration
- Login with email and password
- Role-based access (Admin/User)

🏨 3.2 Hotel Management (Admin)

- Add hotel
- Update hotel details
- Delete hotel
- View hotels

🚪 3.3 Room Management (Admin)

- Add rooms to hotel
- Set price and type
- Assign amenities
- Update/delete rooms

🧩 3.4 Amenity Management

- Add amenities (WiFi, AC, TV, etc.)
- Assign amenities to rooms

🔍 3.5 Search Functionality

- Search by location
- Search by price range
- Search by amenities
- Search by hotel name

📅 3.6 Booking System

- Select room
- Choose check-in and check-out dates
- Create booking
- Update booking status (Confirmed, Cancelled)

📋 3.7 Booking Management

- **Users can**: View their bookings
- **Admin can**: View all bookings

👤 3.8 User Profile

- View user details
- View booking history

🗄️ 4. Data Requirements

4.1 Entities:

- User
- Hotel

- Room
- Booking
- Amenity

4.2 Relationships:

- User → Booking (1:N)
- Hotel → Room (1:N)
- Room → Booking (1:N)
- Room ↔ Amenity (M:N)

🚫 5. Non-Functional Requirements

5.1 Performance

- Fast search results ensuring minimal load times
- Efficient database queries

5.2 Security

- Password encryption (bcrypt)
- Role-based authorization
- Secure APIs

5.3 Usability

- Responsive UI for diverse screen sizes
- Easy navigation

5.4 Scalability

- Modular backend architecture
- RESTful APIs

🔌 6. External Interface Requirements

6.1 User Interface

- Web-based UI (React)
- Mobile responsive design

6.2 API Interface

REST APIs for all operations. Below is the comprehensive list of backend endpoints:

Authentication (/auth)

- POST /auth/register - User registration
- POST /auth/login - User login and token generation

User Management (/users)

- GET /users/me - Fetch logged-in user profile

Hotel Management (/hotels & /admin/hotels)

- GET /hotels - List all hotels
- GET /hotels/{id} - Fetch details of a specific hotel
- POST /admin/hotels - Add a new hotel (Admin only)
- PUT /admin/hotels/{id} - Update hotel details (Admin only)
- DELETE /admin/hotels/{id} - Delete a hotel (Admin only)

Room Management (/rooms & /admin/rooms)

- GET /rooms - List all rooms
- GET /rooms/{id} - Fetch details of a specific room
- POST /admin/rooms - Add a new room to a hotel (Admin only)

Amenities (/amenities)

- GET /amenities - List available amenities

Search (/search)

- GET /search - Search hotels based on dynamic criteria

Bookings (/bookings)

- POST /bookings - Create a new booking
- GET /bookings/my-bookings - Retrieve user's booking history
- PUT /bookings/{id}/cancel - Cancel an existing booking